Trycraz: Diffusion-Based Virtual Try-On with Context-Aware Clothing Recommendation

Darshan Prajapati
Chemical Engineering
Indian Institute of Technology Jodhpur
Roll No: B23CH1032

Sunny Kumar
Mechanical Engineering
Indian Institute of Technology Jodhpur
Roll No: B23ME1073

GitHub repository: https://github.com/CrazAzartto/trycraz-virtual-tryon

Abstract

Virtual try-on shows how a garment would look on a person, but it does not say whether that garment suits the place the person is going to. We built Trycraz, a working system that answers both questions. A user uploads a photo and a garment, and the pretrained CatVTON diffusion model renders the person wearing it. A context-aware recommender ranks a 52-garment catalogue for a given occasion, setting and weather, explains every score in plain words, and sends any pick straight to the try-on. We did not train the generator. Our work is the system around it: input validation, a pose-based quality gate built on MediaPipe, garment background removal with U^2-Net, a job-based GPU server that runs on a free Colab or Kaggle T4, a hybrid recommender that combines hand-written dress-code knowledge with CLIP zero-shot scores, and a Streamlit application. On a T4 a try-on takes 61.5–62.4 s for 30 denoising steps with a peak of 4.89 GB of GPU memory. Our first real runs had two clear failures; diagnosing them one hypothesis at a time showed that a photo cut off at the hips, not the garment, was the cause. After adding a framing rule and background removal, all six validation try-ons succeeded. On twelve labelled scenarios the hybrid recommender reaches a mean P@5 of 0.833 and nDCG@5 of 0.830, higher than either attribute rules (0.767/0.751) or CLIP alone (0.667/0.691).

1 Introduction

When people shop for clothes online, two questions come up again and again: *how will this look on me*, and *is it right for where I am going*. Image-based virtual try-on addresses the first one. Given a photo of a person and a photo of a garment, a try-on model produces a new photo of that person wearing the garment, while keeping their face, body shape, pose and background. Recent diffusion models have made these results realistic enough to be useful. They still say nothing about the second question, so a user can get a convincing picture of themselves in a hoodie without being told that a hoodie is a poor choice for a job interview.

Trycraz combines the two. The user chooses whether they are shopping for menswear or womenswear, then describes the occasion (for example interview, college or wedding), the setting (indoor or outdoor) and the weather. The system ranks a catalogue, explains each score, and lets the user try any recommendation on with one click. The try-on result is shown with a drag-to-compare slider and a verdict on how well the garment suits the chosen occasion.

The image synthesis is done by CatVTON [1], a pretrained model that we use as published. We chose it because it is the lightest of the current diffusion try-on models, handles tops, bottoms and

B.Tech Computer Vision course project, IIT Jodhpur, November 2026.

dresses, and builds its own clothing mask. Everything else in this report is our own work. The main contributions are:

- an end-to-end system that runs a 899 M-parameter diffusion model on a free cloud T4 and serves it to a laptop app over an authenticated tunnel, with progress reporting and clear error messages;
- a pre-generation quality gate that uses pose landmarks to catch photos that are likely to fail, including a rule we derived from a real failure: photos cut off at the hips;
- a hybrid, explainable recommender whose six factor scores produce both the ranking and the explanation, so the two can never disagree;
- an honest evaluation: timing and memory on real hardware, a hypothesis-by-hypothesis failure analysis, ranking metrics with an ablation, and 166 automated tests.

The rest of this report is organised as follows. Section 2 reviews earlier work on virtual try-on, the models we build on, and outfit recommendation. Section 3 states the problem and the constraints we worked under. Section 4 describes the proposed method: the system, the try-on model, our own pipeline around it, the recommender and the optimisations we made. Section 5 reports the experiments and results, including a failure analysis and a discussion of limitations, and Section 6 concludes.

2 Literature Review

Warping-based virtual try-on. Early try-on methods split the problem in two: first warp the garment to the body, then blend it in. VITON-HD [3] raised the resolution to 1024×768 with a GAN and a normalisation layer that handles misalignment between the warped garment and the body. Warping methods still break when the pose is unusual or the garment is loose, because one geometric transform cannot describe how cloth folds.

Diffusion-based virtual try-on. Diffusion models changed this. Latent diffusion [2] runs the denoising process in the compressed latent space of an autoencoder, which makes 1024-pixel images affordable. IDM-VTON [4] builds on two SDXL-sized networks, one for the person and one for the garment, and gives very good results, but it needs far more memory than a free GPU offers. CatVTON [1] takes the opposite route. It removes the second network and the text encoder altogether, and simply places the person and the garment side by side in one image, as described in Section 4.2. Because the person and garment share one self-attention space, CatVTON learns garment placement implicitly. That keeps it small enough for the free hardware available to us, which is the main reason we chose it.

Human parsing, pose and segmentation. Every try-on method needs to know which pixels belong to the clothing that will be replaced. DensePose [5] maps each body pixel to a body-part surface, and Self-Correction for Human Parsing (SCHP) [6] labels pixels as hair, face, upper clothes, trousers and so on. CatVTON combines the two to build its clothing mask. Lighter models are useful outside the generator too: BlazePose [7] finds 33 body landmarks in real time on a CPU, and U^2-Net [8], a nested U-structure for salient object detection, separates a foreground object from its background without class labels. We use these two to check and clean the inputs before any GPU time is spent.

Outfit recommendation and zero-shot vision–language models. Most published outfit recommenders learn compatibility between garments, or between a garment and a setting, from large labelled fashion datasets. We had no such data for occasions and weather, so we wrote the dress-code knowledge by hand. To let the system look at the garment photo itself, we added CLIP [9], a vision–language model trained on image–text pairs, which can score an image against a text description such as "formal clothes for a job interview" without any training on our data. Combining rules with zero-shot scores keeps every decision explainable.

Gap. Try-on systems show how a garment looks but not whether it suits the situation, and recommenders suggest garments without showing them on the user. The open-source try-on demos we tried, including CatVTON's own, stop at the image. Our project connects the two: an explainable, context-aware recommender whose suggestions can be tried on in one step.

3 Problem Statement

We address two coupled tasks.

Task 1: virtual try-on. Given a photo I_p of a person, a photo I_g of a garment and the garment's category $\tau \in \{\text{upper}, \text{lower}, \text{overall}\}$, produce an image $\hat{I}$ in which the same person wears the garment. Outside the region the garment covers, $\hat{I}$ should match I_p: the face, hair, hands, pose, body shape and background must not change. Inside it, the garment's colour, pattern, text and shape should be carried over faithfully and draped to the person's pose.

Task 2: context-aware recommendation. Given a catalogue $\mathcal{G} = \{g_1, \ldots, g_N\}$ of garments and a context $c = (a, o, s, w, p)$, consisting of the wear line a (menswear or womenswear), the occasion o (one of 9), the setting s (indoor or outdoor), the weather w (one of 4) and optional colour, style and formality preferences p, return:

- a ranking of the suitable garments by a score $S(g, c) \in [0, 1]$;
- for every ranked garment, a short explanation that agrees with its score;
- the garments that are clearly unsuitable (for example shorts for an interview), listed separately with a reason.

The user must be able to send any recommended garment straight to Task 1, and to ask whether a garment of their own suits the context.

Constraints. The problem is shaped as much by our constraints as by the tasks:

1. *No local GPU capable of diffusion.* Our laptop has an RTX 3050 with 4 GB of VRAM and 7.6 GB of RAM. The only GPUs available to us are free, time-limited Colab and Kaggle T4 sessions, which can disconnect.
2. *No training data for suitability.* There is no public dataset that labels garments by occasion and weather in a way we could train on, and we have no budget to train or fine-tune a try-on model.
3. *Uncontrolled inputs.* Users take photos at home: partial framing, cluttered backgrounds, poor light. The try-on models we found were trained on studio photos.
4. *Trust.* A recommendation is only useful if the user can see why it was made, so every score must be explainable, and the explanation must not contradict the score.

Success criteria. For Task 1: realistic results on front-facing photos, judged visually, with a response time of about one to two minutes, and memory that fits a T4. For Task 2: ranking quality on labelled scenarios, measured by Precision@5 and nDCG@5, and rankings that change with the context. For the system as a whole: clear messages instead of crashes when inputs or the server go wrong, verified by automated tests.

4 Proposed Method

4.1 System Overview

Fig. 1 shows the system. It is split across two machines because the diffusion model does not fit on our laptop (an RTX 3050 with 4 GB of VRAM and 7.6 GB of system RAM). The laptop runs the Streamlit app, the input checks, background removal, CLIP and the recommender, all on the CPU. The try-on model runs on a free Google Colab or Kaggle notebook with an NVIDIA T4 (16 GB). The notebook starts a small FastAPI server and exposes it through a Cloudflare quick tunnel, so the laptop reaches it over HTTPS with an API key.

A try-on request goes through seven steps. The laptop validates and checks the photos, removes the garment background and uploads both images. The server queues the job, builds the clothing mask, runs the diffusion model and keeps the result until the laptop collects it. The laptop then shows the result next to the original photo and, if the user came from the recommender, how well the garment

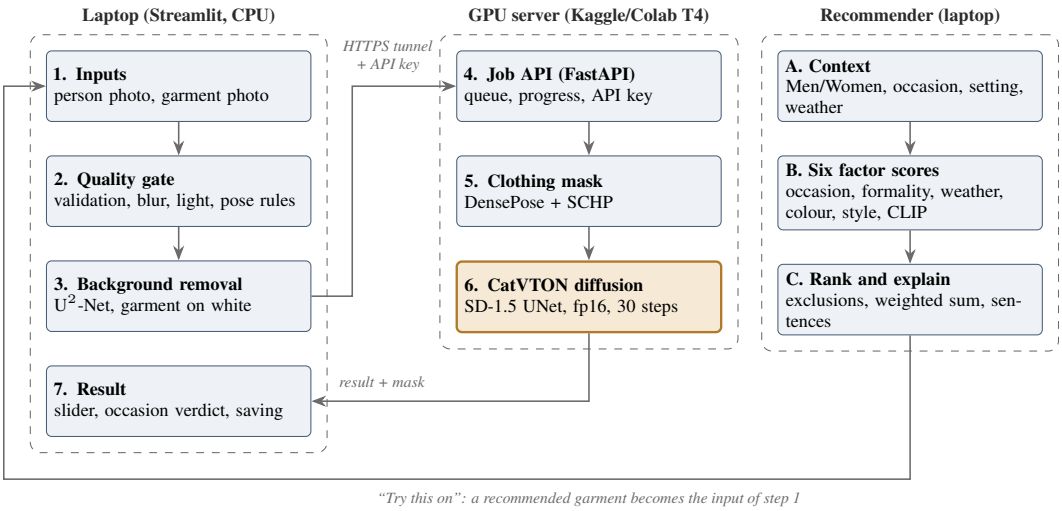


Figure 1: Trycraz architecture. Blue boxes are our code (some call pretrained networks: MediaPipe in step 2, U^2-Net in step 3, CLIP in step B). The orange box is the pretrained CatVTON generator, used unchanged. Steps 1–3 and 7 run on the laptop; steps 4–6 run on a free cloud GPU.

suits the chosen occasion. Every real result is saved with its inputs and settings, so a past run can be shown even when the GPU session is offline.

4.2 Try-On Model: CatVTON

CatVTON [1] is a Stable Diffusion 1.5 inpainting UNet [2] adapted for try-on. It works in four steps.

1. *Mask.* An AutoMasker combines DensePose [5], which labels body parts, with SCHP [6] human parsing on two label sets (LIP and ATR). Simple rules turn these maps into a clothing-agnostic mask: the region to repaint for the chosen garment type, with the face, hair, hands and other garments protected.
2. *Concatenate.* The masked person image and the garment image are encoded by the VAE and placed side by side along the width, giving one latent that is twice as wide.
3. *Denoise.* Starting from noise in the person half, the UNet removes noise step by step with a DDIM sampler. Its self-attention layers let every position in the person half attend to every position in the garment half. This is how the model learns where each part of the garment should go, so no separate warping network is needed.
4. *Decode.* The person half is decoded back to an image. We then copy the original pixels outside the mask back in, so the face and background are not changed at all.

CatVTON has no text encoder and no separate garment encoder, which is why it is small: 899.06 M parameters in total. Its authors froze everything except the self-attention layers, so only 49.57 M parameters were trained. They used the standard noise-prediction objective

$$\mathcal{L} = \mathbb{E}_{z_0,\epsilon,t} \left\| \epsilon - \epsilon_\theta(z_t, t, c) \right\|_2^2 , \tag{1}$$

with the DREAM training strategy ($\lambda = 10$), AdamW at a constant learning rate of 10^{-5}, and dropped the garment condition 10% of the time so that classifier-free guidance can be used. The multi-task checkpoint we use (`mix-48k-1024`) was trained at 1024×768 with batch size 32 for 48,000 steps on about 73k pairs from VITON-HD, DressCode and DeepFashion, on 8 A800 GPUs.

At inference, classifier-free guidance combines a prediction with the garment and one without it,

$$\hat{\epsilon} = \epsilon_\theta(z_t, \varnothing) + s \left(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing) \right) , \tag{2}$$

with guidance scale $s = 2.5$. Larger values push the result harder towards the garment and make the colours more saturated.

Table 1: Try-on models we considered (summary)

Model	Approach	Garments	Drawback for us
CP-VTON	warp + GAN	tops	256×192 output
VITON-HD	warp + GAN	tops	heavy preprocessing
StableVITON	diffusion	tops	no automatic mask
OOTDiffusion	diffusion, 2 UNets	half/full body	two UNets
IDM-VTON	diffusion, 2 SDXL	tops*	GPU memory
CatVTON	**diffusion, 1 UNet**	**all three**	**none critical**

All three = tops, bottoms, dresses. *Upper body on VITON-HD.

Table 2: Quality rules on the 3:4 crop the model sees

Check	Rule	Level
Missing body parts	shoulders (and hips, knees or ankles, by garment type) not visible, $v < 0.5$	error
Cut off at hips	hip line $y_{\text{hip}}/H > 0.85$	error
Low hips	$0.75 < y_{\text{hip}}/H \leq 0.85$	warning
Side view	shoulder width / torso length < 0.35	warning
Back view	left shoulder appears left of right shoulder	warning
Hands on torso	a wrist inside the shoulder–hip box	note
Person too small	visible landmarks span $< 30\%$ of H	warning
Blur	variance of Laplacian < 35 (at 512 px)	warning
Too dark / bright	mean luminance < 60 or > 225	warning

We compared ten try-on models before choosing (Table 1). Older warping methods work at 256×192 or need preprocessing that is hard to run on arbitrary user photos. Of the diffusion models, IDM-VTON and OOTDiffusion use two UNets, and StableVITON has no automatic mask for user photos. CatVTON was the only one that combined 1024×768 output, tops, bottoms and dresses, an automatic mask and a memory footprint that fits a free T4.

4.3 Input Validation and Preprocessing

Every upload is checked before anything else: the file must exist, have an allowed extension, decode as an image, be at most 15 MB, have its short side at least 256 px and long side at most 8000 px, and an aspect ratio of at most 3:1. EXIF rotation is applied and transparent images are flattened onto white. CatVTON works at 768×1024 (3:4). We crop the person photo with exactly the same centre crop that CatVTON applies internally, and a unit test checks that the two agree pixel for pixel. This matters because the quality checks below must look at the same part of the photo that the model will see. Garment images are padded rather than cropped, so a sleeve or hem is never cut off.

4.4 Pose-Based Quality Gate

A try-on takes about a minute of shared GPU time, so it is worth catching bad photos first. We run the MediaPipe Pose Landmarker (BlazePose [7], lite model) on the cropped photo. It returns 33 body landmarks with a visibility score each. On top of these we wrote the geometric rules in Table 2. Errors mean the try-on is very likely to fail; warnings mean it may degrade. Neither blocks the user, because the rules are heuristics and the user may know better.

The hip rule was not in our first design. It came out of the failure analysis in Section 5.4, where photos that ended at the hips produced grey filler below the garment. The blur threshold was first set to 60, but sharp studio photos scored as low as 50, so we recalibrated it on 20 images: sharp originals scored 50–1562 and the same images blurred with $\sigma = 1.5$ px scored 2–28. A threshold of 35 separates the two groups.

4.5 Garment Background Removal

CatVTON was trained on retailer product photos on plain white backgrounds. Our garment photos were taken on beds, floors and doors, and in one run the red bedsheet under a pair of black trousers leaked into the result. We therefore run U^2-Net [8], a salient object segmentation network, on every garment photo. The garment is cut out, placed on white and cropped to its bounding box. The same step is applied when we build the catalogue and when a user uploads a garment, and the user can switch it off.

4.6 Try-On Server

The server wraps CatVTON in a small job API: `POST /jobs` uploads the two images and the settings and returns a job id at once, `GET /jobs/{id}` returns the stage (queued, masking, diffusion, done) and the current denoising step, and `/health` reports the GPU and model state. We count progress by wrapping the scheduler's step function. This job design is not an extra: the Cloudflare quick tunnel closes any request that runs longer than about 100 s, and a try-on plus queueing can take longer than that. One job runs at a time and others wait in the queue. Each error has its own status code (for example 401 for a wrong key, 422 for bad settings, 507 for GPU out-of-memory), which the client turns into a plain message. The notebook is generated from the server source by a script, and a test checks that the notebook contains the current code byte for byte.

4.7 Context-Aware Recommender

Knowledge base. All dress-code knowledge lives in one YAML file: 9 occasions (casual outing, college, office, formal meeting, interview, wedding, party, outdoor sports, travel), 4 weather types (hot, mild, cold, rainy), 26 garment types and 16 colour names. Each occasion has a target formality on a 1–5 scale, preferred styles, discouraged and excluded garment types, an optional colour palette and a CLIP prompt. Each weather type has a target warmth for indoors and outdoors, and flags for breathability or water resistance. These values are our own judgement, not learned from data, and are kept in one file so they can be read and criticised.

Context processing. The user's choices become a target profile. The target formality is the occasion's value shifted by the user's "more casual / more formal" slider and clipped to [1, 5]. The target warmth depends on the setting, and water resistance only counts outdoors.

Scoring. Each garment g gets six factor values $f_k(g, c) \in [0, 1]$ for the context c, and the score is their weighted sum

$$S(g, c) = \sum_k w_k \, f_k(g, c), \qquad \sum_k w_k = 1, \tag{3}$$

with w = 0.30 (occasion), 0.20 (formality), 0.20 (weather), 0.10 (colour), 0.05 (style) and 0.15 (CLIP). The occasion factor is 1 if the garment is tagged as a primary choice for the occasion, 0.6 for a secondary choice, 0.2 otherwise, and 0 for a discouraged type. Formality and weather are distances on their scales:

$$f_{\text{form}} = \max\!\left(0,\, 1 - \tfrac{|\phi_g - \phi^*|}{4}\right), \tag{4}$$

$$f_{\text{weather}} = 0.7 \max\!\left(0,\, 1 - \tfrac{|\theta_g - \theta^*|}{3}\right) + 0.3\, s_g, \tag{5}$$

where ϕ is formality, θ is warmth, the star marks the target, and s_g is 1 if the garment has the property the weather asks for (breathable when hot, water resistant in outdoor rain). Colour uses the user's preferred colours, or the occasion's palette if none are given. When a factor has nothing to judge, for example no style preference, it is set to a neutral 0.5 and left out of the explanation. Hard exclusions remove items before ranking: garments outside the chosen categories or the chosen wear line (unisex items always pass), and types excluded for the occasion, such as shorts for an interview. These are listed separately as "not suitable".

CLIP factor. The attribute factors only know what the catalogue says about a garment. CLIP ViT-B/32 [9] looks at the photo itself. We embed every catalogue image once. For each occasion and weather type we embed its prompt with three templates ("a photo of …", "a product photo of …", "a photo of a garment: …") and average them, which is called prompt ensembling. A softmax over

Table 3: Pretrained models used and where they run

Model	Role in Trycraz	Runs on
CatVTON (SD-1.5)	generates the try-on image	T4 GPU
DensePose (R50)	body-part map for the mask	T4 GPU
SCHP (LIP, ATR)	human parsing for the mask	T4 GPU
BlazePose (lite)	landmarks for the quality gate	laptop CPU
U^2-Net	garment background removal	laptop CPU
CLIP ViT-B/32	occasion score, garment type	laptop CPU

cosine similarities gives probabilities p_{occ} and p_{wx}, and

$$f_{\text{clip}} = \tfrac{1}{2}\left(\frac{p_{\text{occ}}(o)}{\max_{o'} p_{\text{occ}}(o')} + \frac{p_{\text{wx}}(w)}{\max_{w'} p_{\text{wx}}(w')}\right), \tag{6}$$

so a garment scores 1 when CLIP finds it most typical of the chosen occasion and weather. If CLIP cannot be loaded, its weight is shared among the other factors and the app keeps working.

Explanations. Each factor returns its value together with a positive and a negative phrase. A phrase is used only if the value is at least 0.7 (a reason) or at most 0.3 (a caution), and the verdict follows the total: "Recommended" at 0.70 or above, "A reasonable option" at 0.50 or above, and "Weak match" below. Because the sentence is built from the same numbers as the score, it cannot praise a garment for something that scored badly. An early version did exactly that, which is why we made this a tested rule (Section 5.7).

User's own garments. For an uploaded garment we estimate its attributes from the photo. CLIP picks the garment type from 26 classes (with three flat-lay style templates), and the main colours come from k-means ($k = 3$) on the garment pixels in CIELAB space. Pixels within $\Delta E = 18$ of the border colour are treated as background, and cluster centres are named after the nearest of our 16 colours. The estimated garment is then scored like a catalogue item, and the app labels the result as an estimate.

4.8 Supporting Pretrained Models

Table 3 lists every pretrained network in the system. None was retrained. MediaPipe and CLIP run on the laptop CPU; CLIP's first load downloads about 600 MB and takes 149 s once, after which analysing one garment takes 0.25–0.9 s.

4.9 Optimization

Since we did not train any network, there is no optimizer or loss of our own to report. Our optimization work was of three other kinds: making a large model run fast and within memory on free hardware, calibrating the hand-set thresholds on real failures, and keeping the evaluation honest.

Inference settings.

- *Half precision.* The T4 has no fast bfloat16, so all networks run in float16. This halves weight memory, and the measured peak stayed at 4.89 GB of the 15 GB available.
- *Fewer steps.* CatVTON's default is 50 DDIM steps. We use 30, which at about 2.1 s per step takes about 62 s instead of roughly 100 s. All results in this report use 30 steps; we did not run a formal 30-versus-50 quality comparison.
- *Native resolution.* We generate at the model's native 768×1024, because resizing the person photo to another aspect ratio distorts the body.
- *Repaint.* Copying unmasked pixels back costs nothing and guarantees the face and background are untouched.
- *Fixed seed.* Seed 42 makes runs repeatable, which was essential for the diagnosis below.

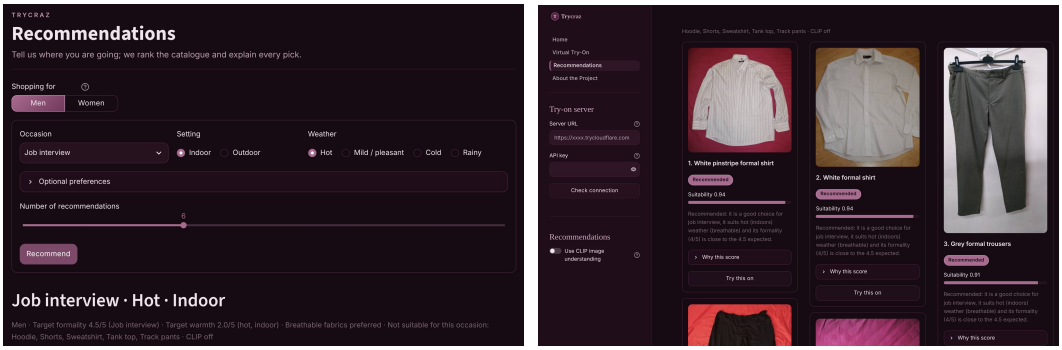


Figure 2: The Recommendations page. Left: the user picks Men or Women, then the occasion, setting and weather (here a job interview, indoors, hot). Right: ranked cards with verdict, score, explanation, a breakdown of the six factors and a button that opens the garment in the try-on page.

System-level choices. The job API with polling exists because of the tunnel's request limit. Model weights are loaded once per session (86 s with a warm cache) and kept in GPU memory. On the laptop, CLIP embeddings of the catalogue are computed once and cached, so ranking 52 garments takes well under a second, and Streamlit caches the quality checks and background removal for each uploaded image. All library versions and the CatVTON commit are pinned, which mattered because newer PyTorch changed the default of `torch.load` and broke SCHP's checkpoint loader; we load only the official checkpoints with the old behaviour.

Calibration and the weights. The two thresholds that matter most were set from measurements, not guessed: the blur threshold from 20 sharp and blurred images, and the hip threshold from the photos that failed (hips at 0.89 and 0.91 of the image height) and the ones that worked (0.43–0.50). The recommender weights, on the other hand, were set by hand *before* the evaluation and were deliberately not tuned on the evaluation labels. With only twelve scenarios, tuning on them would mostly fit the noise in our own labels and make the reported numbers look better than they are.

4.10 Catalogue and Application

We built a 52-garment demo catalogue from the CC0 Clothing Dataset by A. Grigorev: 36 tops and jackets, 10 bottoms and 6 dresses, of which 39 are unisex and 13 womenswear. Each item has a type, try-on category, formality, warmth, breathability, water resistance, colours, styles, primary and secondary occasions and a wear line. Annotations were made by hand and are marked as such. Sample person photos are from Unsplash (free licence).

The Streamlit app has four pages: Home, Virtual Try-On, Recommendations and About the Project. Recommendations first asks whether the user is shopping for men or women; without this step the list mixed both. Each result card shows the garment, the verdict, the score, a one-line explanation, a "Why this score" table of the six factors and a "Try this on" button that opens the try-on page with that garment and its type already selected (Fig. 2). The try-on page shows the quality report before generation, a progress bar during it, and afterwards a comparison slider, the occasion verdict, downloads and the region the model repainted. A section on the Recommendations page lets the user upload their own garment and see whether it suits the chosen context.

5 Experiments and Results

5.1 Setup

Try-on ran on a free Colab T4 with CatVTON at a pinned commit, `mix-48k-1024` weights, fp16, 768×1024, 30 DDIM steps, guidance 2.5, seed 42 and repaint on. The client was a Windows laptop (Intel i5-12450H, 7.6 GB RAM) connected over the tunnel. Try-on quality was judged by eye, since we have no ground truth image for a new person–garment pair.

Table 4: Try-on performance on a T4 (10 runs, 30 steps)

Measure	Value
Diffusion, 30 steps	61.5–62.4 s (≈2.1 s/step)
Mask (DensePose + 2×SCHP)	0.94–1.36 s
Peak GPU memory	4.89 GB of 15 GB
Round trip from the laptop	69–133 s
Model load, once per session	86 s

Table 5: Real try-on runs, judged by eye

Run	Garment (photo background)	Type	Result
R2	navy blazer (door, pink wall)	upper	failure
R3	white formal shirt (wooden floor)	upper	failure
R4	white wrap blouse (white wall)	upper	success
R5	black trousers (red bedsheet)	lower	partial
R6	light blue jeans (beige sheet)	lower	success
R7	tropical print dress (wooden floor)	overall	partial
V1–V6	six catalogue garments (bg. removed)	all	6/6 success

5.2 Speed and Memory

Table 4 summarises ten runs. The diffusion loop is almost all of the server time; masking takes about a second. The round trip seen by the user varies more because it includes uploading, the tunnel and waiting in the queue behind another job.

5.3 Try-On Quality

Our first six real runs with home-style garment photos (R2–R7 in Table 5) gave two successes, two partial results and two failures. After the two fixes described next, we ran six validation try-ons with full-body photos and background-removed catalogue garments, and all six succeeded (Fig. 3). The blazer keeps its lapels and button, the elephant print on the T-shirt survives, the jeans keep their fading, and in every case the face, hands, shoes and background are unchanged. The remaining flaws are small: the blazer comes out slightly lighter than the original, the white shirt looks a little sheer, and the T-shirt's sleeves end higher than the sweater's did, so the model has to invent the forearms.

5.4 Failure Diagnosis

Runs R2 and R3 put a blazer and a formal shirt on the same man, and both came out as a grey, knitted looking top with a pale halo around the shoulders (Fig. 4a–c). We tested one hypothesis at a time, keeping the seed and settings fixed (Table 6).

The photo of this man ends at the hips, so the region to repaint touches the bottom edge of the image. For a long garment there is no body below the hem to anchor it, and the model fills the missing part with grey. We turned this into the `cut_off_at_hips` rule in Table 2 and moved the photo out of our samples into a folder of failure examples. A second problem showed up in R5: the black trousers had been photographed on a red bedsheet, and red patches appeared inside the generated trousers (Fig. 4j). Background removal fixed this (l), which is why we kept it even though it was not the cause of R2.

5.5 Recommendation Quality

We wrote twelve test scenarios, each a context plus graded relevance labels for the catalogue (2 = good choice, 1 = acceptable, unlisted = unsuitable). The labels were written from the garment photos and names *before* the recommender was run on this catalogue. With $k = 5$, Precision@5 is the share

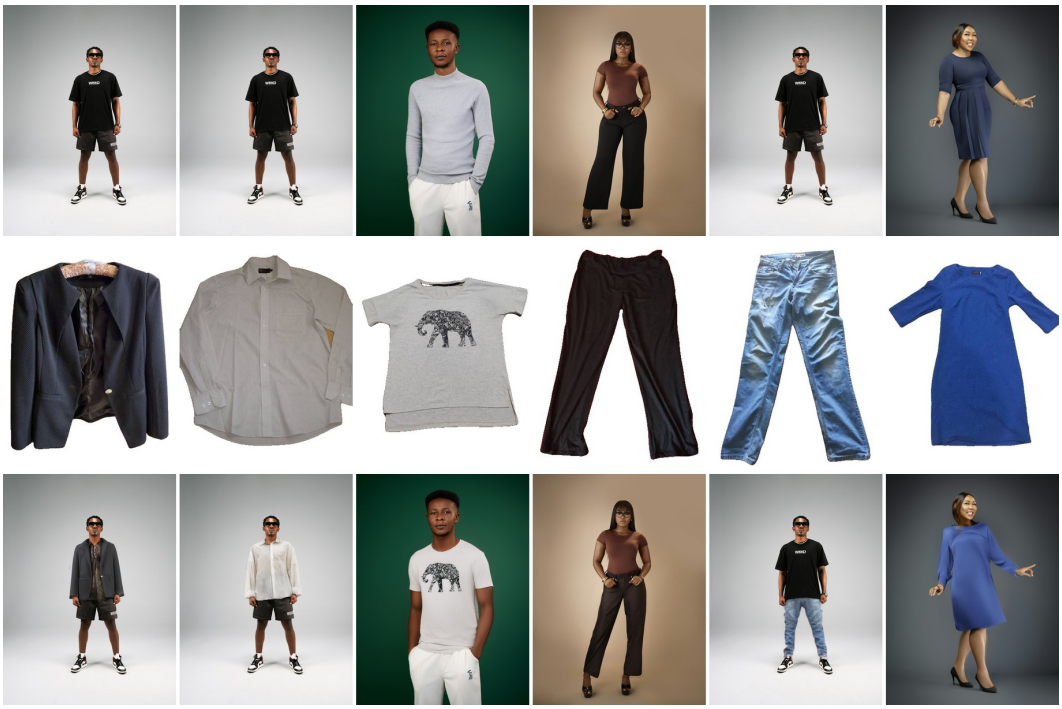


Figure 3: Validation try-ons V1–V6 generated by CatVTON on a T4 through our pipeline. Rows: input person, garment after our background removal, result. Columns: navy blazer, white formal shirt, printed T-shirt (all upper body), black formal trousers, light blue jeans (lower body) and a royal blue sheath dress (full outfit).

Table 6: Diagnosing R2/R3: hypothesis, test, evidence

Hypothesis	Test	Evidence
Cluttered garment background	remove it with U^2-Net, rerun	still grey with a halo (d): rejected
The garment is the problem	same blazer on two other people	correct blazer both times (e, f): rejected
The person photo is the problem	a T-shirt that worked before, on this man	top half fine, bottom half grey (g): confirmed
Why this photo?	MediaPipe hip height on the 3:4 crop	failing: 0.89, 0.91; working: 0.43–0.50

of the top five with relevance at least 1, and

$$\text{nDCG@}k = \frac{1}{\text{IDCG@}k} \sum_{i=1}^{k} \frac{2^{r_i} - 1}{\log_2(i+1)}, \tag{7}$$

where r_i is the relevance of the item at rank i and IDCG is the same sum for the ideal order, so nDCG also rewards putting the best items first.

Table 7 shows the ablation and Fig. 5 the per-scenario results. The hybrid beats both of its parts on average. The two signals fail in different places. The attribute rules are weak on College tops (nDCG 0.39), where a rain jacket and a camisole reach the top five because their occasion tags are broad, and on Party (0.28). CLIP alone is weak on Wedding (0.31) and Office dresses (0.33), where it cannot tell a smart dress from a casual one in a flat photo, but it is perfect on Travel in the rain, where a rain jacket simply looks like a rain jacket. Combined, College rises to 0.69 and Wedding to 0.62.

To check that the ranking really depends on the context, we computed Kendall's τ between the rankings of every pair of scenarios that share at least five garments (45 pairs). The mean is 0.01 for



(a) input (b) garment (c) R2 result (d) bg. removed (e) other person (f) other person (g) known-good tee



(h) input (i) on a red sheet (j) R5 result (k) bg. removed (l) V3 result

Figure 4: Failure analysis. Top: the man's photo ends at the hips. The blazer fails on him with or without its background (c, d), works on two other people (e, f), and even a T-shirt that worked before fails on him in the lower half (g). Bottom: the red bedsheet behind the trousers leaks into the result (j); after background removal the trousers come out clean (l).

Table 7: Recommender ablation (12 scenarios, $k = 5$, real CLIP)

Configuration	Mean P@5	Mean nDCG@5
Attributes only (no CLIP)	0.767	0.751
CLIP only	0.667	0.691
Hybrid (default weights)	**0.833**	**0.830**

the hybrid (0.07 for attributes only), where $\tau = 1$ would mean the same order whatever the occasion. Party is the weakest scenario for both the hybrid (0.20) and the rules (0.28). "Party wear" covers very different garments, and our catalogue has few of them. In four scenarios one signal alone ranks better than the mix, which suggests the weights are reasonable but not optimal; we chose not to tune them on these labels (Section 4.9).

5.6 Automatic Garment Annotation

For uploaded garments the attributes come from the photo. We compared the automatic estimates with our manual labels on all 52 catalogue garments (Table 8). Background removal mainly helps colour, because bedsheet and floor pixels no longer enter the colour clusters. Garment type is harder than it looks: several classes are genuinely close (polo and T-shirt, jacket and blazer, sweater and sweatshirt).

5.7 Software Testing

The project has 166 automated tests, and all pass on the laptop and on Linux. They cover input validation with broken and unusual files, the crop arithmetic against CatVTON's own function, the server's job life-cycle and every error code, the laptop client against a real local server, the pose rules on synthetic landmarks, the recommender's behaviour in typical scenarios, the metrics against hand-computed values and the four app pages. One test generates explanations for all 9 occasions and 4 weather types and checks that no stated reason contains negative wording, which guards the bug mentioned in Section 4.7. CatVTON's own code was also run end to end with randomly initialised weights on the CPU, which confirmed that it works on Python 3.12 before we spent any GPU time. Against the real server, a wrong API key, an unreachable URL and a queued second request all produced the intended messages.

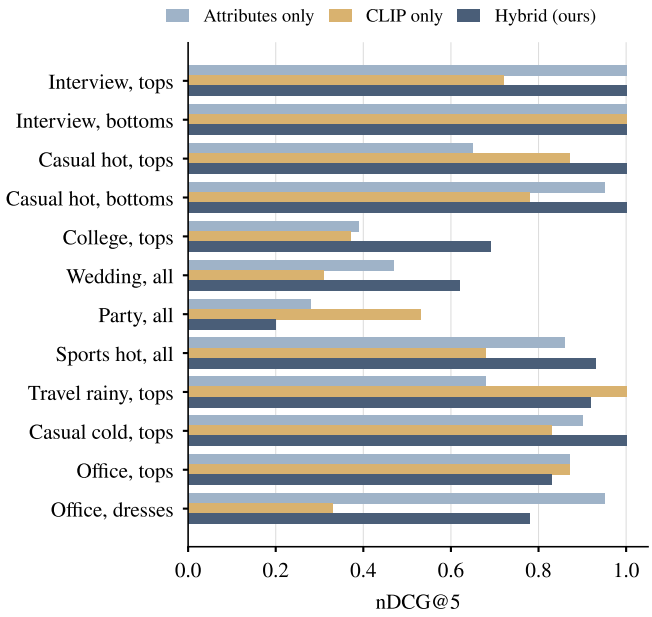


Figure 5: nDCG@5 per scenario for the three configurations. The hybrid is best or tied in 8 of 12 scenarios. In Party, Travel, Office tops and Office dresses one signal alone does better.

Table 8: Automatic vs. manual annotation (52 garments)

Measure	Before bg. removal	After
Type, CLIP top-1 (26 classes)	32/52 (62%)	33/52 (63%)
Main colour, k-means name	26/52 (50%)	30/52 (58%)

5.8 Discussion and Limitations

What worked. The split between a laptop and a free cloud GPU made a 899 M-parameter model usable without any lab hardware, at the cost of about a minute per image and a session that has to be restarted every few hours. The most useful part of the try-on pipeline turned out to be the checks *before* generation. CatVTON itself was reliable once its inputs looked like its training data: a front-facing person with body visible below the garment, and a garment on a plain background. Both of our failures were input problems, and both are now caught or fixed automatically.

Why the hybrid helps. Rules and CLIP make different mistakes. The rules know conventions that cannot be seen in a photo, such as shorts not being worn to an interview, but they rely on our tags. CLIP sees the actual garment but is easily fooled by photo style. A weighted sum lets each correct the other, and because the explanation comes from the same factor values, the user can see which signal drove each decision.

Limitations. We should be clear about what our numbers do and do not show.

- Try-on quality was judged by eye on twelve runs. We have not yet computed paired metrics such as SSIM, LPIPS or KID on the VITON-HD test set; for reference, the CatVTON authors report SSIM 0.870 and LPIPS 0.057 there [1].
- The recommender labels come from a single annotator who also wrote the knowledge base and the catalogue tags, so the scores are probably optimistic. Twelve scenarios and 52 garments are small.
- The catalogue images are only 400 px wide, which limits fine detail, and there is no traditional Indian wear such as kurtas or sarees. Draped garments like sarees are outside what this model supports.

- The dress-code rules are hand-written and culturally specific. The app presents its output as suggestions, not rules.
- A free cloud GPU can disconnect, and each new session spends a few minutes downloading and loading the model.

Licences. CatVTON is released under CC BY-NC-SA 4.0, so Trycraz is for academic, non-commercial use. All photos we used are CC0 or under the Unsplash licence, and credits are listed in the app.

Future work. The next steps follow from the limitations: run the paired VITON-HD metrics on a T4; have a second person label the scenarios independently and report the agreement; photograph our own clothes on a plain background to replace the low-resolution catalogue images; and add a short user study in which people rate realism and suitability.

6 Conclusions

Trycraz shows that realistic virtual try-on and context-aware recommendation can be combined in one working system built on free hardware. A pretrained CatVTON model generates the images on a cloud T4 in about 62 s with under 5 GB of GPU memory, while our own pipeline decides whether a photo is good enough to send, cleans the garment photo and handles the long-running job over an unreliable tunnel. Diagnosing our failures step by step showed that input framing, not the garment or the model, caused them, and after adding a framing rule and background removal all six validation try-ons succeeded. The hybrid recommender, which adds CLIP's view of the photo to hand-written dress-code rules, reached a mean P@5 of 0.833 and nDCG@5 of 0.830, better than either part alone, and explains every score from the same numbers that produced it. The main gaps are evaluation gaps, namely paired image metrics and independent labels, and they are the first things we would add.

Code availability. The full source code, setup guide, test suite and evaluation scripts are public at `https://github.com/CrazAzartto/trycraz-virtual-tryon`.

References

[1] Z. Chong *et al.*, "CatVTON: Concatenation is all you need for virtual try-on with diffusion models," in *Proc. ICLR*, 2025.

[2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE CVPR*, 2022, pp. 10684–10695.

[3] S. Choi, S. Park, M. Lee, and J. Choo, "VITON-HD: High-resolution virtual try-on via misalignment-aware normalization," in *Proc. IEEE CVPR*, 2021, pp. 14131–14140.

[4] Y. Choi, S. Kwak, K. Lee, H. Choi, and J. Shin, "Improving diffusion models for authentic virtual try-on in the wild," in *Proc. ECCV*, 2024.

[5] R. A. Güler, N. Neverova, and I. Kokkinos, "DensePose: Dense human pose estimation in the wild," in *Proc. IEEE CVPR*, 2018, pp. 7297–7306.

[6] P. Li, Y. Xu, Y. Wei, and Y. Yang, "Self-correction for human parsing," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 6, pp. 3260–3271, 2022.

[7] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.

[8] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U^2-Net: Going deeper with nested U-structure for salient object detection," *Pattern Recognit.*, vol. 106, p. 107404, 2020.

[9] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.